Crossing the Streams: SSH Plaintext Recovery via a Common Compression Context in Multiplexed Channels

Fabian Bäumer
Ruhr University Bochum
Bochum, Germany
fabian.baeumer@rub.de

Marcus Brinkmann
Ruhr University Bochum
Bochum, Germany
marcus.brinkmann@rub.de

A Artifact Appendix

A.1 Abstract

The artifacts implement the adaptive compression attack in Section 4 for the three scenarios in Section 5: direct plaintext injection into a network-exposed port forward, browser-based injection into a loopback-bound forward, and recovery of an Ansible `become` password from a session channel. A Docker Compose testbed and Python scripts provide end-to-end verification of each scenario and benchmarks with randomly generated passwords. The artifacts include the raw measurements for Table 3 and are licensed under the Apache-2.0 open-source license.

Generative AI usage. The proof-of-concepts in these artifacts and the text of this appendix were drafted with Claude and ChatGPT and then manually reviewed and revised by the authors. The paper's Generative AI Usage statement gives the details.

A.2 Description & Requirements

The repository's `README.md` describes the architecture, configuration, control APIs, and execution procedures.

- `attacker/attack/` contains the attack engine, configuration, and adapters for the three scenarios. `attacker/mitm.py` implements the TCP forwarder, packet capture, and HTTP control API.
- `client/` and `server/` contain the victim client and SSH server, including the Ansible configuration and playbook.
- `scripts/` contains the verification scripts, `benchmark.py`, the commit-margin sweep script `sweep_commit_margin.sh`, and `stats.py` for analysis.
- `evaluation/` contains the measurements for Table 3, organized by scenario and configuration.
- The testbed is defined in `docker-compose.yml`; `docker-compose.bench.yml` configures parallel benchmark stacks.

A.2.1 Security, privacy, and ethical concerns. The experiments perform plaintext recovery within a Docker bridge network on the evaluator's host. They use generated SSH keys and synthetic passwords: either `hunter2` or values from a seeded pseudorandom number generator. The experiments require no external targets or real credentials. SSH keys are generated in `keys/`.

The attacker container is granted `NET_ADMIN` and `NET_RAW` capabilities for network capture. The client and attacker control APIs are published on host ports 8000 and 9000. These unauthenticated APIs can initiate attacks and inject data into the victim's tunnel. Access to these ports must therefore be restricted to trusted hosts.

A.2.2 How to access. Version 3 is archived on Zenodo at https://doi.org/10.5281/zenodo.22308155. The development repository is https://github.com/RUB-NDS/Crossing-the-Streams-Artifacts; tag `v3` identifies the corresponding release.

A.2.3 Hardware dependencies. No specialized hardware is required. For the verification experiments, we recommend an x86-64 Linux host with at least 4 cores, 8 GB of RAM, and 5 GB of free disk space for the container images. Experiment (E4) computes the Table 3 statistics from the included data and requires only a host with Python 3. Collecting new measurements is optional; the number of parallel stacks can be reduced to fit the available hardware, at the cost of longer execution time.

The original measurements used 100 parallel Compose projects (400 running containers) on a dual-socket AMD EPYC 7763 system with 2 TB of RAM running Ubuntu 22.04. This configuration supported parallel data collection and is not required for artifact evaluation.

A.2.4 Software dependencies. The host requires Linux, Docker Engine with the Compose plugin, and Python 3. The host scripts use only the Python standard library. The parallel benchmark runs used Ubuntu 22.04. The host tool versions reported for the evaluation are Docker 29.8, Compose v5.5, and Python 3.14. Container builds install the remaining dependencies, including `aiohttp`, Scapy, redispy, Playwright with headless Firefox, and `ansible-core`. Python package versions are pinned in the requirements files. The base images are `python:3.14-slim`, `debian:bookworm-slim`, and `redis:8-alpine`. Building the images requires Internet access.

A.2.5 Benchmarks. No external dataset is required. The benchmark generates passwords using the seed specified by `--seed`. The Table 3 measurements used seed 42 for direct and browser injection and seed 4253 for Ansible. The recorded measurements are included in `evaluation/`. Passwords with other characters, such as digits or symbols, were not evaluated for Table 3. The benchmark option `--alphabet` selects another set of single-byte characters. Each measurement round tests every remaining candidate, so the number of guesses grows with the alphabet size.

A.3 Set Up

A.3.1 Installation. Clone the repository or extract the Zenodo archive, then run the following command from the repository root:

```
docker compose up -d --build
```

The build takes several minutes, including the Firefox download. The testbed comprises five services: key generation, SSH server, Redis, victim client, and attacker. In `docker compose ps -a`, `poc-keygen` should have exited with code 0 and the other four services should be running.

A.3.2 Basic test. The attack engine's unit tests run on the host without Docker:

```
python -m attacker.attack.tests.test_config
```

Replace `test_config` with each of the following module names for the remaining tests:

```
test_engine_helpers
test_engine_expected
test_alignment
```

Each test prints a line ending in ok and exits with code 0. The modules test the conversion of JSON configuration overrides (`test_config`), the selection of alignment lengths and trimming of the known prefix (`test_engine_helpers`), the early abort on a byte that differs from the expected value (`test_engine_expected`), and the generation of alignment bytes (`test_alignment`).

Check the running testbed through its control APIs:

```
curl -s localhost:8000/status
curl -s localhost:9000/status
```

The client should report `ssh_connected: true`, zlib compression in both directions, and an active `redis_tunnel`. The attacker should report the forwarding target, capture interface, and BPF filter. Both APIs should report `browser_connected: true`.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1): A network attacker with access to a forwarded TCP port can recover a secret transmitted over another channel of the same SSH connection by injecting chosen plaintext and observing ciphertext lengths. Experiment (E1) evaluates direct plaintext injection (Section 5.1, Fig. 3).

(C2): Under the combined eavesdropper and web attacker model, JavaScript in the victim's browser can inject guesses into a loopback-bound port forward to recover the secret. Experiment (E2) evaluates browser-based injection (Section 5.2, Fig. 4).

(C3): The attack also recovers Ansible's `become` password, which is transmitted through a session channel. Experiment (E3) evaluates this scenario (Section 5.3, Fig. 5).

(C4): Table 3 reports the guess counts for recovering 8-character lowercase passwords across three scenarios and five noise-compensation configurations. All 100 trials per evaluated combination succeeded at the reported commit margins, with up to two retries per password. Experiment (E4) evaluates these results (Section 5.4), excluding the two combinations marked `n/a`.

A.4.2 Experiments. Experiments (E1)–(E3) each recover `hunter2` and must be run sequentially, in any order. **Only one attack may run at a time in each testbed; do not run the three verification scripts concurrently.** Each script prints the recovered password and exits with code 0 on success. All commands below are run from the repository root. Note that the compute times for (E1)–(E3) depend on the host and measurement noise.

Starting or Recreating a Testbed. Before restarting or retrying an attack, stop the verification script if it is still running, then recreate the testbed:

```
docker compose down
docker compose up -d
```

Wait for the basic test checks to pass before starting the script again. Stopping a verification script alone does not ensure that the attack in the container has stopped. Additional guidance on how to handle failures and timeouts is given at the end of this section. Track attack progress, candidate margins, and completed bytes with:

```
docker compose logs -f attacker
```

For parallel benchmarks, use `docker logs -f` with the individual attacker container name, e.g., `bench-0-attacker` for the first stack with the default benchmark prefix.

(E1): *Direct plaintext injection* [2 person-minutes + 2–5 compute-minutes]. Recover the password from a Redis authentication request by injecting guesses into the forwarded TCP port.
Preparation: Start the testbed as described under Installation.
Execution: `python scripts/verify_direct.py`
Results: The script checks API availability, SSH compression, the Redis tunnel, and packet capture. It recovers the password length and contents, reporting guess counts and elapsed time. The script prints `VERIFICATION PASSED` on success and exits with code 1 if the recovered password is incorrect. The guess counts are not directly comparable to Table 3: the verification uses lowercase letters and digits, the adaptive alignment sweep with candidate elimination, and commit margin 16, and typically needs 2,000–3,000 guesses for `hunter2`.

(E2): *Browser-based plaintext injection* [2 person-minutes + 17–70 compute-minutes]. Recover the Redis password through a loopback-bound forward using the attacker's page loaded in the victim's headless Firefox.
Preparation: Start the testbed and wait for the browser to connect. Check that `browser_connected` is `true` in the attacker status.
Execution: `python scripts/verify_browser.py`
Results: The script checks the prerequisites of (E1) and the browser's WebSocket control channel, then recovers the password. It prints `Status: PASS` on success. HTTP framing introduces additional measurement noise, addressed by the alignment sweep and a higher commit margin. E2 has the largest runtime variation; the script waits up to 7200 seconds for each attack phase.

(E3): *Ansible password recovery* [2 person-minutes + 4–7 compute-minutes]. Recover the `become` password written to the SSH client's standard input, using a new playbook execution for each guess.
Preparation: Start the testbed.
Execution: `python scripts/verify_ansible.py`
Results: The script checks the `LocalForward` configuration, sets the sudo password, and verifies that Ansible writes it to standard input. It recovers the length byte in the channel-data header and the password, then prints `Status: PASS` on success.

(E4): *Table 3* [15 person-minutes + 1 compute-minute for the included data]. Compute the statistics in Table 3 from the

included measurements. Optionally, collect new measurements for individual configurations.

Preparation: Analyzing the included data requires only Python 3. New measurements require the container images and SSH keys generated during Installation.

Execution: For each evaluated combination, run:

```
D=evaluation/<scenario>/<config>
python scripts/stats.py \
  $D/benchmark_results_<scenario>_cm<M>.json
```

Directory names are lowercase. Replace `<M>` with the Table 3 commit margin: 8, 40, 16, 16, 80 for direct injection under NO, FS, AS, CE, AS+CE; 48, 64, 88 for browser injection under FS, AS, AS+CE; and 8 for all Ansible configurations. Browser injection under NO and CE is marked `n/a` and has no data. Files for lower margins contain earlier steps of the sweep.

To collect new measurements for Ansible under NO, run:

```
python scripts/benchmark.py --stacks 20 \
  --trials 100 --scenarios ansible \
  --compensation NO --alignment-length 1 \
  --commit-margin 8 --seed 4253 \
  --max-retries 2
```

Reduce `--stacks` to fit the available hardware. Direct and browser injection use `--seed 42`; direct injection under NO and CE uses `--alignment-length 2`. `scripts/sweep_commit_margin.sh` automates the sweep.

Results: `stats.py` reports the number of successful trials, minimum, maximum, mean, median, and standard deviation of their guess counts. For the included files at the reported margins, these statistics match Table 3 and `n` is 100. New measurements are written to `benchmark_results.json`. The benchmark exits with code 0 if all trials succeed, 1 if any fail, and 2 for an infrastructure failure.

New measurements are expected to recover all 100 passwords and produce comparable guess counts and relative performance across configurations. Exact counts may vary with measurement noise. Measuring all 13 combinations at the reported margins took approximately 23 hours with 100 parallel stacks. Individual combinations ranged from 11 minutes (Ansible, NO) to 6 hours (browser, FS).

Troubleshooting. Measurement noise can cause recovery failures. If the prerequisite checks pass but recovery fails, recreate the testbed as described above and rerun the verification script. **For repeated failures, edit the `config` dictionary** sent to `/run_attack`: in `verify_direct.py`, this is in `_run_attack()`; in `verify_browser.py`, it is in `browser_attack()`; in `verify_ansible.py`, edit both `phase1_body` and `phase2_body`. These scripts are in `scripts/`, and changes do not require rebuilding the images. First increase `commit_margin` in steps of 8, starting from 16 for direct injection, 64 for browser injection, or 8 for Ansible. This is the minimum margin required to select a candidate byte. If the logs report exhausted rounds before reaching the margin, increase `max_rounds` (e.g., double its current value: 128 for direct/browser injection, 96 in the Ansible verification script).

If packet capture appears delayed under load, increase `settle`, the delay in seconds between measurement steps, from its default of 0.01, 0.05, or 0.25 for direct, browser, or Ansible injection, respectively; doubling it provides an initial adjustment. If Ansible's fixed alignment produces no distinguishing signal, set `alignment_mode` to `full_sweep` and `alignment_lengths` to `list(range(8))` in both phases. These adjustments increase execution time and may increase guess counts. If a script times out while the container logs still show progress, recreate the testbed and rerun the script with a larger HTTP request timeout using `--timeout SECONDS`. The defaults are 3600, 7200, and 1800 seconds for direct, browser, and Ansible injection, respectively.

A.5 Notes on Reusability

The attack engine separates candidate ranking and noise compensation from the scenario adapters. Each adapter implements `measure_once()`, which accepts a known prefix, candidate byte, and alignment bytes and returns the observed byte count. Adapters handle compression-context flushing, channel setup, secret transmission, guess injection, and packet measurement. Additional scenarios can be implemented through this interface and registered with `/run_attack`; the benchmark requires corresponding scenario support.

The `/run_attack` API accepts configuration overrides for the noise-compensation strategies, commit margin, alphabet, known prefix, and alignment lengths. The README maps these keys to the paper's notation. The default alignment sweep covers lengths 0–7 for the 8-byte padding granularity of ChaCha20-Poly1305. AES-based modes require lengths 0–15, increasing the number of measurements (Section 8.1). The client status API reports the negotiated cipher. The buffer flush uses 32 KiB, corresponding to deflate's maximum LZ77 window; smaller compression windows permit shorter flushes.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.